

ENTERPRISE SECURITY ASSESSMENT LAB

WEB APPLICATION VULNERABILITY ASSESSMENT & PENETRATION TESTING REPORT

Professional Portfolio Edition - Based on Authorized DVWA Lab Testing

Prepared By	Jagriti Banerjee
Repository	Enterprise-Security-Assessment-Lab
LEGAL AND ETHICAL NOTICE This report documents only authorized lab testing performed inside a private, intentionally vulnerable environment. The techniques described must not be used against public, third-party, or production systems without explicit written permission.	

Table of Contents

1. Executive Summary
2. Engagement Overview
3. Scope and Rules of Engagement
4. Testing Environment
5. Tools and Technology Arsenal
6. Methodology and Testing Framework
7. Reconnaissance and Attack Surface Mapping
8. Vulnerability Summary Dashboard
9. Detailed Findings
10. OWASP Top 10 Coverage Matrix
11. Attack Chain and Risk Narrative
12. Remediation Roadmap
13. Evidence and Screenshot Mapping
14. Redaction and GitHub Safety Checklist
15. Conclusion
16. Appendix
17. Document Control

1. Executive Summary

This report presents the Web Application Vulnerability Assessment and Penetration Testing work completed as part of the Enterprise Security Assessment Lab. The Web VAPT module was designed to demonstrate a realistic, evidence-based web application security testing workflow in a safe, controlled, and authorized environment.

The primary target used for this assessment was DVWA, an intentionally vulnerable web application deployed locally through Docker. The assessment demonstrates practical ability in reconnaissance, enumeration, web scanning, manual request analysis, vulnerability validation, exploitation proof inside a lab, evidence capture, remediation planning, and professional reporting.

This document intentionally avoids fake production claims. It does not claim testing against real client infrastructure, public websites, production applications, or third-party systems. All findings are framed as lab-validated issues with real-world risk context.

1.1 High-Level Assessment Result

Assessment Area	Result	Evidence Type
Lab environment prepared	Completed	Kali and Docker screenshots
Vulnerable web target deployed	Completed	DVWA browser evidence
Reconnaissance performed	Completed	Passive recon and Nmap evidence
Enumeration performed	Completed	Gobuster and Nikto evidence
Manual request analysis performed	Completed	Burp Suite evidence
Vulnerabilities validated	Completed	SQLi, XSS, CMDi, upload, CSRF evidence
Evidence prepared for GitHub	Completed	Screenshot mapping and redaction checklist

1.2 Key Validated Vulnerability Classes

ID	Vulnerability Class	Lab Status	Real-World Risk Style
WEB-001	SQL Injection	Validated	Critical
WEB-002	Database Enumeration via SQLMap	Validated	Critical
WEB-003	Weak Hash Cracking	Validated	High
WEB-004	Reflected Cross-Site Scripting	Validated	High
WEB-005	Stored Cross-Site Scripting	Validated	High
WEB-006	OS Command Injection	Validated	Critical
WEB-007	Insecure File Upload / Webshell	Validated	Critical
WEB-008	Cross-Site Request Forgery	Validated	Medium
WEB-009	Directory and File Enumeration	Completed	Medium / Informational

WEB-010	Technology Fingerprinting	Completed	Informational
WEB-011	Web Server Scan Findings	Completed	Informational / Medium

Risk Summary

In a production environment, the combined presence of SQL Injection, command injection, weak credential storage, and unrestricted file upload could enable database compromise, credential exposure, server-side code execution, and broader application compromise. In this project, all activity was conducted only inside a deliberately vulnerable lab.

2. Engagement Overview

2.1 Engagement Objectives

1. Deploy and access a vulnerable web application lab.
2. Verify required tools and testing environment readiness.
3. Perform passive and active reconnaissance.
4. Enumerate hidden directories and files.
5. Fingerprint application technologies and server behaviour.
6. Scan for common web misconfigurations.
7. Intercept and modify HTTP requests using Burp Suite.
8. Validate SQL Injection manually and with SQLMap.
9. Demonstrate weak hash cracking risk.
10. Validate reflected and stored Cross-Site Scripting.
11. Validate OS command injection in a controlled lab.
12. Validate insecure file upload and webshell-style impact.
13. Demonstrate CSRF request forgery.
14. Map evidence to screenshots for GitHub documentation.
15. Provide remediation guidance for each validated issue.

2.2 Engagement Type

Field	Details
Engagement Type	Web Application Vulnerability Assessment and Penetration Testing
Testing Mode	Black-Box to Grey-Box Lab Testing
Target Type	Intentionally vulnerable local web application
Primary Target	DVWA
Attack / Testing Machine	Kali Linux
Deployment Method	Docker
Evidence Type	Screenshots and selected tool outputs
Report Type	Professional GitHub portfolio technical report

3. Scope and Rules of Engagement

3.1 In-Scope Assets

Asset	Description	Status
Kali Linux	Primary security testing machine	In scope
Docker	Used to host the vulnerable web target	In scope
DVWA	Primary vulnerable web application	In scope
Browser	Used for manual testing and screenshots	In scope
Burp Suite	Used for interception and Repeater testing	In scope
Localhost / Private Lab	Authorized lab network and application scope	In scope

3.2 Out-of-Scope Activities

Activity	Reason
Testing public websites	Not authorized
Testing real production applications	Not part of the lab
Attacking third-party systems	Illegal and unethical
Denial-of-Service testing	Not required for this project
Malware deployment	Not part of VAPT scope
Persistence or backdoor installation	Excluded for safety
Publishing raw credentials or hashes	Prohibited
Uploading unredacted database dumps	Prohibited

4. Testing Environment

The Web VAPT lab was performed using Kali Linux as the testing machine and DVWA as the intentionally vulnerable application target. Docker was used to run the web application target locally, while browser-based manual testing and Burp Suite interception were used to validate the behaviour of the application.

```

Kali Linux Testing Machine
  Tools: Nmap, Gobuster, Nikto, WhatWeb, Burp Suite, SQLMap, John, Browser
        |
        | HTTP / Local Lab Traffic
        v
Docker Vulnerable Web Target
  Application: DVWA
  Purpose: Web vulnerability testing lab
  
```

4.1 Environment Evidence

Evidence	Screenshot File
Kali Linux desktop	01-kali-linux-desktop-full-screen.png
Docker containers running	02-docker-containers-running.png
DVWA login page accessible	03-dvwa-login-page-browser.png
Tool version check	04-tools-installed-version-check.png
Installation/setup evidence	installed.png

5. Tools and Technology Arsenal

Tool	Purpose in This Assessment
Kali Linux	Security testing operating system
Docker	Running the local vulnerable target
Nmap	Port and service discovery
Gobuster	Directory and file enumeration
Nikto	Web server vulnerability and misconfiguration scanning
WhatWeb	Technology fingerprinting
Burp Suite	HTTP interception, request analysis, and Repeater testing
SQLMap	SQL Injection validation and database enumeration
John the Ripper	Password hash cracking demonstration
Browser / DevTools	Manual testing, validation, and evidence capture

6. Methodology and Testing Framework

The methodology followed a structured penetration testing lifecycle adapted for an authorized lab project. The goal was to demonstrate the same discipline used in a professional assessment while avoiding unauthorized or destructive behaviour.

Phase 1 - Lab setup
 Phase 2 - Tool verification
 Phase 3 - Passive reconnaissance
 Phase 4 - Active reconnaissance
 Phase 5 - Directory enumeration
 Phase 6 - Technology fingerprinting
 Phase 7 - Vulnerability scanning
 Phase 8 - Manual request analysis
 Phase 9 - SQL Injection testing
 Phase 10 - SQLMap validation
 Phase 11 - Hash cracking
 Phase 12 - XSS testing
 Phase 13 - Command Injection testing
 Phase 14 - File Upload testing
 Phase 15 - CSRF testing
 Phase 16 - Evidence capture

6.1 Frameworks Referenced

Standard / Framework	Application in Report
OWASP Top 10	Web application vulnerability classification
OWASP Web Security Testing Guide	General web testing methodology
PTES	Penetration testing phase structure
CVSS v3.1	Severity reference for real-world context
CWE	Software weakness classification reference

7. Reconnaissance and Attack Surface Mapping

7.1 Passive Reconnaissance

Passive reconnaissance was performed to understand the target application before deeper testing. In this lab context, passive recon included reviewing visible pages, application behaviour, browser responses, headers, and accessible functionality.

Evidence	Screenshot File
Passive reconnaissance part 1	Passive-Recon-p1.png
Passive reconnaissance part 2	Passive-Recon-p2.png
Passive reconnaissance part 3	Passive-Recon-p3.png

7.2 Active Reconnaissance with Nmap

Nmap was used to identify exposed services and validate the reachable attack surface. Service discovery is important even in web testing because exposed ports and supporting services can change the security posture of the application.

Evidence	Screenshot File
Nmap full port scan output	05-nmap-full-port-scan-output.png

7.3 Directory and File Enumeration

Gobuster was used to enumerate directories and files. Directory enumeration helps identify hidden application paths, administrative areas, backups, configuration files, or test resources that may not appear in normal navigation.

Evidence	Screenshot File
Gobuster directory results	06-gobuster-directory-results.png
Additional Gobuster evidence	06-gobuster-directory-results(M).png
Directory and file enumeration summary	Directory-&-File-Enumeration-with-Gobuster.png

7.4 Technology Fingerprinting

WhatWeb was used to identify server and application technologies. Fingerprinting supports vulnerability research, version review, and context-aware testing.

Evidence	Screenshot File
WhatWeb fingerprinting part 1	08-whatweb-tech-fingerprint-p1.png
WhatWeb fingerprinting part 2	08-whatweb-tech-fingerprint-p2.png
WhatWeb fingerprinting part 3	08-whatweb-tech-fingerprint-p3.png
WhatWeb fingerprinting part 4	08-whatweb-tech-fingerprint-p4.png

7.5 Vulnerability Scanning

Nikto was used to perform web server scanning. Automated findings were treated as leads requiring manual review rather than final proof by themselves.

Evidence	Screenshot File
Nikto scan findings	07-nikto-scan-findings.png

8. Vulnerability Summary Dashboard

ID	Finding	Category	Risk Style	Status
WEB-001	SQL Injection	Injection	Critical	Validated
WEB-002	SQLMap Database Enumeration	Injection Impact	Critical	Validated
WEB-003	Weak Hash Cracking	Credential Security	High	Validated
WEB-004	Reflected XSS	Injection	High	Validated
WEB-005	Stored XSS	Injection	High	Validated
WEB-006	OS Command Injection	Injection	Critical	Validated
WEB-007	File Upload / Webshell	File Upload / RCE Risk	Critical	Validated
WEB-008	CSRF Forged Request	Broken Access Control	Medium	Validated
WEB-009	Directory Enumeration	Recon / Exposure	Medium / Info	Completed
WEB-010	Technology Fingerprinting	Recon	Informational	Completed
WEB-011	Nikto Scan Findings	Misconfiguration	Info / Medium	Completed

8.1 Severity Distribution

Risk Style	Count
Critical	4
High	3
Medium	2
Informational	2

Severity Interpretation

Severity is described as a real-world risk style because the target is intentionally vulnerable. Formal CVSS scoring would require production context, asset value, exploitability, and business impact.

9. Detailed Findings

WEB-001 - SQL Injection

Field	Details
Severity Style	Critical in a real-world context
Evidence	09-dvwa-sqli-vulnerable-input-response.png
Status	Validated in authorized lab

SQL Injection occurs when user-controlled input is inserted into a database query without proper validation or parameterization. In this lab, SQL Injection was manually validated using the DVWA SQL Injection module.

Potential Real-World Impact

- Authentication bypass
- Database enumeration
- Data extraction
- Credential hash exposure
- Modification or deletion of records

Recommended Remediation

16. Use parameterized queries and prepared statements.
17. Never concatenate user input into SQL queries.
18. Validate input server-side.
19. Use least-privileged database accounts.
20. Disable verbose database errors.
21. Monitor abnormal query behaviour.

WEB-002 - SQLMap Database Enumeration

Field	Details
Severity Style	Critical in a real-world context
Evidence	10-sqlmap-database-enumeration.png; 11-sqlmap-users-table-dump.png

Status	Validated in authorized lab
--------	-----------------------------

After manual SQL Injection validation, SQLMap was used in the authorized lab to demonstrate the practical impact of SQL Injection. SQLMap was able to enumerate database information and dump lab user data.

Potential Real-World Impact

- Database name discovery
- Table enumeration
- User record extraction
- Credential hash disclosure
- Sensitive data compromise

Recommended Remediation

22. Fix the SQL Injection root cause.
23. Use prepared statements.
24. Restrict database permissions.
25. Perform regression testing after remediation.
26. Rotate credentials if exposure occurred.

WEB-003 - Weak Hash Cracking

Field	Details
Severity Style	High in a real-world context
Evidence	07-john-md5-hashes-cracked.png
Status	Validated in authorized lab

The assessment demonstrated weak hash cracking using John the Ripper. This shows how password hashes exposed through a vulnerable application can be cracked if weak algorithms or weak passwords are used.

Potential Real-World Impact

- Account takeover
- Credential reuse
- Privilege escalation
- Password spraying against other services

Recommended Remediation

27. Use Argon2, bcrypt, or PBKDF2.
28. Use unique salts.
29. Enforce strong password policies.
30. Detect common and leaked passwords.
31. Rotate exposed passwords.

WEB-004 - Reflected Cross-Site Scripting

Field	Details
Severity Style	High in a real-world context
Evidence	15-reflected-xss-alert-popup.png; 15-reflected-xss-alert-popup-p2.png

Status	Validated in authorized lab
--------	-----------------------------

Reflected XSS occurs when user input is immediately reflected into the response without proper output encoding. The lab validated reflected XSS using a harmless browser alert payload.

Potential Real-World Impact

- JavaScript execution in victim browsers
- Phishing
- Session theft if cookies are not protected
- Content manipulation

Recommended Remediation

32. Encode output based on context.
33. Sanitize user input.
34. Use safe templating engines.
35. Implement Content Security Policy.
36. Set cookies with HttpOnly, Secure, and SameSite.

WEB-005 - Stored Cross-Site Scripting

Field	Details
Severity Style	High in a real-world context
Evidence	16-stored-xss-payload-source.png
Status	Validated in authorized lab

Stored XSS occurs when malicious input is stored by the application and later rendered to users. Stored XSS can affect every user who views the stored content.

Potential Real-World Impact

- Persistent JavaScript execution
- Session compromise
- Administrator targeting
- Stored malicious content delivery

Recommended Remediation

37. Sanitize stored input.
38. Encode output when rendering.
39. Use Content Security Policy.
40. Use safe HTML sanitization libraries.
41. Limit allowed input types.

WEB-006 - OS Command Injection

Field	Details
Severity Style	Critical in a real-world context
Evidence	17-command-injection-id-whoami-output.png
Status	Validated in authorized lab

OS Command Injection occurs when user input is passed into system-level commands without safe validation. In the lab, command execution was validated using identity commands.

Potential Real-World Impact

- Arbitrary command execution
- Web server compromise
- File disclosure
- Reverse shell risk
- Full host compromise

Recommended Remediation

42. Avoid shell execution with user input.
43. Use safe APIs instead of command-line calls.
44. Apply strict allowlists.
45. Run services with least privilege.
46. Monitor suspicious command execution.

WEB-007 - Insecure File Upload / Webshell Validation

Field	Details
Severity Style	Critical in a real-world context
Evidence	18-file-upload-webshell-confirmed-p1.png; 18-file-upload-webshell-confirmed-p2.png
Status	Validated in authorized lab

The file upload functionality was tested to validate whether dangerous files could be uploaded and executed. The lab confirmed webshell-style behaviour in the controlled environment.

Potential Real-World Impact

- Remote code execution risk
- Sensitive file access
- Persistent access
- Application compromise

Recommended Remediation

47. Restrict allowed file extensions.
48. Validate MIME type and file content.
49. Rename uploaded files.
50. Store uploads outside the web root.
51. Disable script execution in upload directories.
52. Scan uploaded files.

WEB-008 - Cross-Site Request Forgery

Field	Details
Severity Style	Medium in a real-world context
Evidence	19-csrf-forged-request-proof.png
Status	Validated in authorized lab

CSRF occurs when an attacker causes an authenticated user browser to submit an unwanted request. In the lab, a forged request proof was captured.

Potential Real-World Impact

- Unwanted account changes
- Sensitive action abuse
- Settings modification
- Potential account takeover if password change is affected

Recommended Remediation

53. Use anti-CSRF tokens.
54. Set SameSite cookie attributes.
55. Validate Origin and Referer headers.
56. Require re-authentication for sensitive actions.
57. Use framework-level CSRF protection.

10. OWASP Top 10 Coverage Matrix

OWASP Category	Coverage	Evidence / Notes
A01 Broken Access Control	Partially tested	CSRF and request manipulation evidence
A02 Cryptographic Failures	Partially tested	Weak hash cracking workflow
A03 Injection	Strongly tested	SQLi, XSS, Command Injection
A04 Insecure Design	Reviewed	DVWA intentionally vulnerable workflows
A05 Security Misconfiguration	Tested	Nikto and WhatWeb evidence
A06 Vulnerable and Outdated Components	Reviewed	Technology fingerprinting evidence
A07 Identification and Authentication Failures	Partially tested	Weak/default credential and hash cracking workflow
A08 Software and Data Integrity Failures	Tested	File upload/webshell validation
A09 Security Logging and Monitoring Failures	Not deeply tested	Not claimed
A10 Server-Side Request Forgery	Not tested in Web module	Not claimed

11. Attack Chain and Risk Narrative

The lab demonstrates how individual web vulnerabilities can combine into a more serious compromise path. This is important because real attackers rarely stop after one finding; they chain weaknesses to increase access and impact.

```
Step 1: Reconnaissance
-> Step 2: Directory and file enumeration
-> Step 3: SQL Injection validation
-> Step 4: SQLMap database enumeration
-> Step 5: User table dump and hash exposure
-> Step 6: Weak hash cracking
-> Step 7: Command Injection validation
-> Step 8: File upload / webshell validation
-> Step 9: Application compromise demonstrated in lab
```

11.1 Real-World Business Impact

- Data breach and exposure of user records
- Account takeover and credential reuse
- Administrative compromise
- Server-side command execution
- Service disruption
- Regulatory and compliance impact
- Reputation damage and customer trust loss

12. Remediation Roadmap

Priority	Finding	Recommended Timeline
P0 Immediate	SQL Injection	24-48 hours
P0 Immediate	OS Command Injection	24-48 hours
P0 Immediate	File Upload / Webshell	24-48 hours
P1 Urgent	Stored XSS	3-7 days
P1 Urgent	Reflected XSS	3-7 days
P1 Urgent	Weak Hash Storage	7 days
P2 Medium	CSRF	7-14 days
P2 Medium	Directory Exposure	14-30 days
P3 Low	Header and fingerprinting improvements	30-60 days

12.1 Strategic Recommendations

58. Implement a Secure Software Development Lifecycle.
59. Add security testing to CI/CD pipelines.
60. Perform code review for injection flaws.
61. Use SAST and DAST tools.
62. Perform dependency scanning.
63. Use secure password storage.
64. Apply least privilege everywhere.
65. Harden web server configuration.
66. Use centralized logging and alerting.
67. Conduct regular penetration tests.

13. Evidence and Screenshot Mapping

Evidence Category	Screenshot File
Kali environment	01-kali-linux-desktop-full-screen.png
Docker target	02-docker-containers-running.png
DVWA login	03-dvwa-login-page-browser.png
Tools installed	04-tools-installed-version-check.png
Nmap scan	05-nmap-full-port-scan-output.png
Gobuster	06-gobuster-directory-results.png
Nikto	07-nikto-scan-findings.png
John cracking	07-john-md5-hashes-cracked.png
WhatWeb	08-whatweb-tech-fingerprint-p1.png
SQL Injection	09-dvwa-sqli-vulnerable-input-response.png
Recon reports	09-recon-report-files-generated.png
SQLMap DB enumeration	10-sqlmap-database-enumeration.png
SQLMap users dump	11-sqlmap-users-table-dump.png
Burp intercept	13-burp-intercepted-request.png
Burp Repeater	14-burp-repeater-modified-request-response.png
Reflected XSS	15-reflected-xss-alert-popup.png
Stored XSS	16-stored-xss-payload-source.png
Command Injection	17-command-injection-id-whoami-output.png
File Upload	18-file-upload-webshell-confirmed-p1.png
CSRF	19-csrf-forged-request-proof.png

14. Redaction and GitHub Safety Checklist

Before publishing screenshots or outputs to GitHub, sensitive values must be blurred, cropped, or replaced with placeholders.

Sensitive Item	Action Required
PHPSESSID / cookies	Blur or crop
Authorization headers	Blur completely
Session tokens	Blur completely
Database dumps	Blur sensitive rows and fields
Password hashes	Blur or redact
Cracked passwords	Never publish unredacted
Webshell filenames	Blur if sensitive
Local usernames and paths	Blur where private
Private IPs	Blur if privacy is required

14.1 Files That Should Not Be Uploaded

```
raw database dumps
raw hash files
cracked password files
webshell source files
session cookies
.env files
private keys
sqlmap dump folders
```

15. Conclusion

The Web Application VAPT module successfully demonstrates a complete practical web security testing workflow. It covers environment preparation, reconnaissance, enumeration, scanning, manual validation, exploitation proof inside a safe lab, evidence collection, remediation planning, and professional reporting.

This section is suitable for a cybersecurity portfolio because it shows both technical execution and reporting maturity. It avoids exaggerated production claims and clearly separates lab validation from real-world production risk.

16. Appendix

16.1 Glossary

Term	Meaning
VAPT	Vulnerability Assessment and Penetration Testing
DVWA	Damn Vulnerable Web Application
SQLi	SQL Injection
XSS	Cross-Site Scripting
CSRF	Cross-Site Request Forgery
RCE	Remote Code Execution
OWASP	Open Worldwide Application Security Project
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration

16.2 Learning References

- OWASP Top 10
- OWASP Web Security Testing Guide
- OWASP SQL Injection Prevention Cheat Sheet
- OWASP XSS Prevention Cheat Sheet
- OWASP CSRF Prevention Cheat Sheet
- OWASP File Upload Cheat Sheet
- PortSwigger Web Security Academy

17. Document Control

Version	Date	Description
v1.0	June 2026	Initial professional PDF report created for Web VAPT portfolio module
v1.1	June 2026	Adapted to actual Enterprise Security Assessment Lab evidence
v1.2	June 2026	Formatted as professional public portfolio PDF

End of Report

Enterprise Security Assessment Lab - Web Application VAPT - Professional Portfolio Report